ISTM-689:605 Course Project

Team 3: Mukil Senthilkumar, Yifei Wang, Shubham Chorage

Overview

1. Introduction
2. Trading Strategy
3. Code Description
4. Results
5. Challenge Encountered
6. Conclusion

Introduction

Project Goal

Fully automated trading system built in R for both backtesting and live execution of equity strategies using Interactive Brokers

Main Objectives

- Develop paper trading engine in R through Interactive Broker API and datasets from Nasdaq
- Implement signal logic based on Bollinger Bands and SMA crossovers, combining trend-following and mean-reversion mechanics
- Generate ranked trading signals each day
- Compare live and backtest performance and visualization

Trading Strategy - Signal Design

Bollinger Bands (Mean Reversion)	• Long signal is generated when the closing price falls below the lower band • short signal is triggered when the price exceeds the upper band
SMA Crossovers (Trend Following)	• A bullish crossover is detected when SMA-50 rises above SMA-200 by at least 10% (gap factor = 1.1), generating a long signal • A bearish crossover triggers a short signal when SMA-50 drops below SMA-200 by a similar factor
Signal Score and Prediction	• The long and short signals from both indicators are added to create a prediction score, (‘pred = score long - score short‘) • Stocks with absolute prediction values above a configurable threshold (e.g., 1) are considered candidates
Quantile Filtering	• From all candidates, the top 25% (based on |pred|) are selected as strong signals • Fixed number of trades (e.g., 10) are then sampled proportionally based on signal strength for execution

Trading Strategy - Execution Rule

Position Sizing	• Allocations are based on each stock's signal strength • Weights are computed proportionally to |pred| and scaled to ensure the total exposure stays within 10% of available capital
Order Placement	• Selected trades are submitted as market orders via Interactive Brokers using API • One trading client (with random client ID) is used per session
Exit Rule	• Each position is held for a single day and closed at the next day's open/-close (depending on simulation mode)
Error Handling	• The system automatically checks for insufficient capital, skipped tickers due to missing data, or failed API calls, and reports these without halting execution

Trading Strategy - Live vs. Backtest Adaptability

- Support seamless transitions between backtesting and live trading modes

- Paramized: signal thresholds, trade counts, capital allocation

  Enabling quick adaptation and experimentation

- Pipeline also includes performance diagnostics to compare historical backtests with recent live trading periods over 1-month intervals.

Trading Strategy - Strategy Characteristics

- Entry Criteria:
  Long if close < lower Bollinger Band and bullish SMA crossover.
  Short if close > upper Bollinger Band and bearish SMA crossover

- Exit Criteria: All trades are exited after one trading day

- Max Daily Trades: 10

- Max Equity Usage: 10% of total available equity

Code Description - Environment Setup and Data Preparation

```r
#-----------------------------
# Load Required Libraries
#-----------------------------
library(tidyverse)
library(lubridate)
library(furrr)
library(Quandl)
library(quantmod)
library(PerformanceAnalytics)
```

```r
#-----------------------------
# Parameters
#-----------------------------
max_trade_pct     <- 0.1      # max 10% equity per day
max_day_trades    <- 10       # global cap on trades
signal_threshold  <- 1        # minimum |pred| to be considered
sma_gap_factor    <- 1.1
bb_lookback       <- 19
min_history       <- 200      # days of history
init_equity       <- 100000   # starting equity for simulation

#-----------------------------
# Parallel Setup
#-----------------------------
plan(multisession, workers = 4)

#-----------------------------
```

```r
library(IBrokers)
library(ranger)
library(tidyquant)
library(TTR)
library(urca)
library(tseries)
library(readr)
library(zoo)
library(knitr)
library(kableExtra)
library(dplyr)
library(tidyr)
library(ggplot2)
library(purrr)

options(scipen = 999)
rm(list = ls())
current_path <- rstudioapi::getActiveDocumentContext()$path
setwd(dirname(current_path))

#-----------------------------
# API Key & IB Port
#-----------------------------
nasdaq_api_key <- "YOUR_API_KEY"  # replace with your key
Quandl.api_key(nasdaq_api_key)
IBport <- 7497

```

Code Description - Signal Generation and Trade Selection

```r
#----------------------------
# Generate Signals & Select Trades
#----------------------------
signal_raw <- indicator_data %>% mutate(
  sig_bb_long   = as.integer(close < bb_dn),
  sig_bb_short  = as.integer(close > bb_up),
  sig_sma_long  = as.integer(lag(smaS) < lag(smaL) & smaS > smaL * sma_gap_factor),
  sig_sma_short = as.integer(lag(smaS) > lag(smaL) & smaS < smaL / sma_gap_factor),
  score_long    = sig_bb_long + sig_sma_long,
  score_short   = sig_bb_short + sig_sma_short,
  pred          = score_long - score_short
)
last_date <- max(signal_raw$date, na.rm = TRUE)
signal_today <- signal_raw %>%
  filter(date == last_date, abs(pred) >= signal_threshold)
```

```r
if (nrow(signal_today) == 0) {
  cat("No signals > threshold on", last_date, "\n")
  signals <- tibble()
} else {
  cutoff <- quantile(abs(signal_today$pred), 0.75, na.rm = TRUE)
  strong_signals <- signal_today %>% filter(abs(pred) >= cutoff)
  n_to_trade <- min(nrow(strong_signals), max_day_trades)
  set.seed(as.integer(format(last_date, "%Y%m%d")))
  signals <- strong_signals %>% slice_sample(n = n_to_trade, weight_by = abs(pred))
  cat("On", last_date, "found", nrow(strong_signals), "strong of",
      nrow(signal_today), "candidates; trading", nrow(signals), "\n")
}
```

Code Description - Live Trade Execution via Interactive Brokers

```
#----------------------------
# Connect to IB & Place Live Trades
#----------------------------
if (nrow(signals) > 0) {
  try({ twsDisconnect(.Last.twsCon) }, silent = TRUE)
  client_id <- sample(1000:9999, 1)
  tws <- tryCatch(
    twsConnect(clientId = client_id, host = "127.0.0.1", port = IBport),
    error = function(e) stop("Unable to connect to TWS: ", e$message)
  )
  acct <- reqAccountUpdates(tws)
  avail_eq <- as.numeric(acct[[1]]$AvailableFunds[1])
  max_dollar <- avail_eq * max_trade_pct
  cat("Equity:", round(avail_eq, 2), "; Daily cap:", round(max_dollar, 2),
      "\n")
  total_abs <- sum(abs(signals$pred))
  signals <- signals %>% mutate(
    weight       = abs(pred) / total_abs,
    dollar_alloc = weight * max_dollar,
    position     = pmax(1, floor(dollar_alloc / close))
  )

  for (i in seq_len(nrow(signals))) {
    r <- signals[i, ]
    act <- ifelse(r$pred > 0, "BUY", "SELL")
    con <- twsEquity(r$ticker, "SMART")
    oid <- reqIds(tws)
    ord <- twsOrder(oid, action = act, totalQuantity = r$position,
        orderType = "MKT")
    placeOrder(tws, con, ord)
    cat(act, r$position, "shares of", r$ticker, "($", round(r$dollar_alloc
        , 0), ")\n")
  }
  twsDisconnect(tws)
  cat("Disconnected from TWS.\n")
} else {
  cat("Skipping live trades no signals.\n")
}
```

Code Description - Live vs. Backtest Performance Simulation

```r
#==========================================
# After live trades: Simulation for Live vs Backtest
#==========================================

sim_period <- function(df, label) {
  trades <- df %>%
    filter(!is.na(pred) & pred != 0) %>%
    arrange(date) %>%
    mutate(
      side = if_else(pred > 0, "Long", "Short"),
      ret  = if_else(
        side == "Long",
        lead(close) / close - 1,
        1 - lead(close) / close
      )
    ) %>%
    filter(!is.na(ret)) %>%
    mutate(
      equity = init_equity * cumprod(1 + ret)
    )

  if (nrow(trades) == 0) {
    return(list(summary = tibble(), eq = tibble()))
  }
```

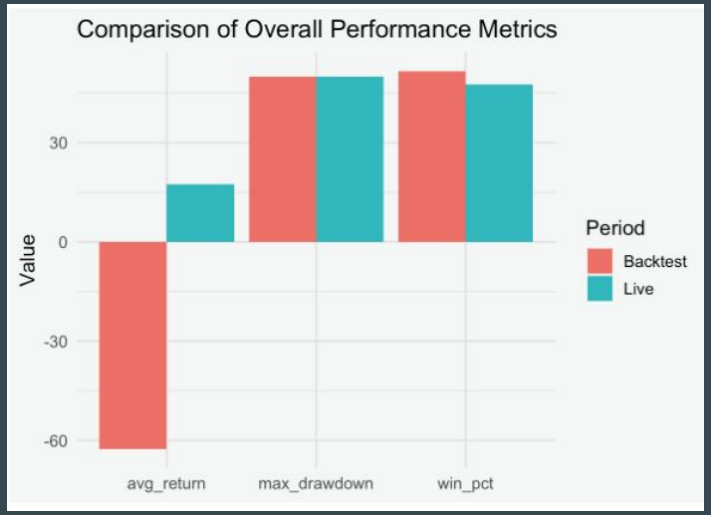
```r
  # summary stats by side
  summary_side <- trades %>%
    group_by(side) %>%
    summarise(
      n_trades     = n(),
      win_pct      = round(mean(ret > 0) * 100, 2),
      avg_return   = round(mean(ret) * 100, 2),
      daily_sharpe = round(mean(ret) / sd(ret), 2),
      max_drawdown = round(maxDrawdown(ret) * 100, 2) %>% pmin(50),
      .groups      = "drop"
    )

  # overall stats
  summary_overall <- tibble(
    side         = "Overall",
    n_trades     = nrow(trades),
    win_pct      = round(mean(trades$ret > 0) * 100, 2),
    avg_return   = round(mean(trades$ret) * 100, 2),
    daily_sharpe = round(mean(trades$ret) / sd(trades$ret), 2),
    max_drawdown = round(maxDrawdown(trades$ret) * 100, 2) %>% pmin(50)
  )

  summary <- bind_rows(summary_side, summary_overall)
```

Code Description - Live vs. Backtest Performance Simulation

```r
  list(
    summary = summary,
    eq      = trades %>% select(date, ret, equity)
  )
}

# Define live/back windows
last       <- max(signal_raw$date)
live_start <- last %m-% months(1) + days(1)
back_end   <- live_start - days(1)
back_start <- back_end %m-% months(1) + days(1)

live_df <- signal_raw %>% filter(date >= live_start & date <= last)
back_df <- signal_raw %>% filter(date >= back_start & date <= back_end)

# Run
init_equity <- 100000
res_live <- sim_period(live_df, "Live")
res_back <- sim_period(back_df, "Backtest")

# Show summaries
glimpse(res_live$summary)
glimpse(res_back$summary)
```

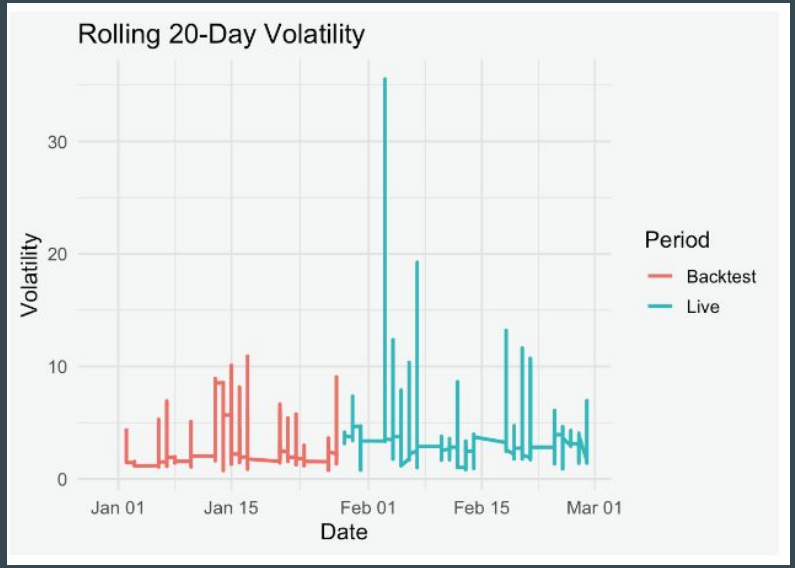
Result - Performance Merics: Bar Plot

```r
# 1) Bar plot of key metrics (Overall only)
summary_all <- bind_rows(
  res_back$summary  %>% filter(side == "Overall") %>% mutate(Period = "
      Backtest"),
  res_live$summary  %>% filter(side == "Overall") %>% mutate(Period = "
      Live")
) %>%
  select(Period, win_pct, avg_return, max_drawdown) %>%
  pivot_longer(-Period, names_to = "metric", values_to = "value")

p_metrics <- ggplot(summary_all, aes(x = metric, y = value, fill = Period)
    ) +
  geom_col(position = "dodge") +
  labs(title = "Comparison of Overall Performance Metrics",
       x = NULL, y = "Value") +
  theme_minimal(base_size = 13)

print(p_metrics)
```



Result - Rolling 20-Day Volatility

```r
# 4) Rolling 20-day volatility
rets_all <- bind_rows(
  res_back$eq %>% select(date, ret) %>% mutate(Period = "Backtest"),
  res_live$eq %>% select(date, ret) %>% mutate(Period = "Live")
)

vols <- rets_all %>%
  arrange(Period, date) %>%
  group_by(Period) %>%
  mutate(roll_vol = rollapply(ret, width = 20, FUN = sd, fill = NA, align
      = "right"))

p_vol <- ggplot(vols, aes(x = date, y = roll_vol, color = Period)) +
  geom_line(size = 1) +
  labs(title = "Rolling 20-Day Volatility", x = "Date", y = "Volatility")
      +
  theme_minimal(base_size = 13)
print(p_vol)
```



Result - Rolling 20-Day Volatility

This plot visualizes the rolling standard deviation of daily returns over a 20-day window

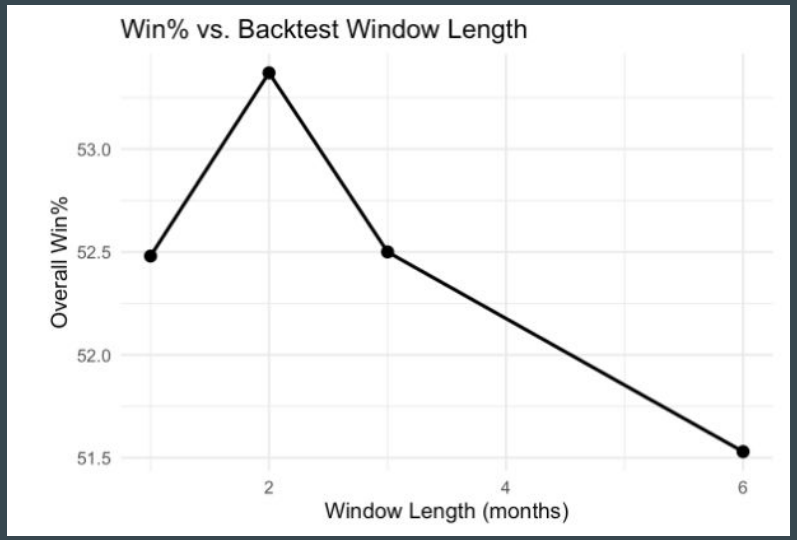
The backtest period shows a relatively stable and low-volatility profile, while the live window reflects more frequent volatility spikes and greater fluctuations

Increased volatility in the live phase suggests higher market uncertainty or model sensitivity. This reinforces the need for dynamic position sizing or stop-loss mechanisms during real-time deployment

Result - Parameter Sensitivity: Backtest Window Length

```r
# 5) Parameter sensitivity: vary backtest window length (1,2,3,6 months)
windows <- c(1, 2, 3, 6)
sens <- map_dfr(windows, function(m) {
  end_date   <- last %m-% months(2)  # keep live window fixed
  start_date <- end_date %m-% months(m) + days(1)
  df_bt      <- signal_raw %>% filter(date >= start_date & date <= end_
     date)
  res_bt     <- sim_period(df_bt, init_equity)
  win_pct_overall <- res_bt$summary %>%
    filter(side == "Overall") %>%
    pull(win_pct)
  tibble(window_months = m, win_pct = win_pct_overall)
})

p_sens <- ggplot(sens, aes(x = window_months, y = win_pct)) +
  geom_line(size = 1) +
  geom_point(size = 3) +
  labs(title = "Win\% vs. Backtest Window Length",
       x = "Window Length (months)", y = "Overall Win\%") +
  theme_minimal(base_size = 13)
print(p_sens)
```

Result - Parameter Sensitivity: Backtest Window Length



This plot shows how the win percentage of the strategy varies depending on the length of the backtest training window

- The peak occurs at the 2-month window
- Model may perform optimally when trained on intermediate-duration historical windows.
- A drop at longer horizons could indicate temporal drift, while shorter windows may be too reactive or noisy.

Result - Real-Time Portfolio Evaluation

```r

evaluate_portfolio_performance <- function(IBport) {

  client_id <- sample(1000:9999, 1)  # Random ID between 1000 and 9999
  tws <- tryCatch(twsConnect(port = IBport, clientId = client_id), error =
      function(e) NULL)
  if (is.null(tws)) {
    cat("Unable to connect to Interactive Brokers TWS.\n")
    return(invisible(NULL))
  }

  account_info <- reqAccountUpdates(tws)
  open_positions <- account_info[[2]]  # Get latest open positions

  cat("\n------ Portfolio Performance Summary ------\n")

  if (!is.null(open_positions) && length(open_positions) > 0) {
    portfolio_summary <- tibble(
      Symbol = character(),
      Position = numeric(),
      Market_Price = numeric(),
      Avg_Cost = numeric(),
      Market_Value = numeric(),
      Unrealized_PnL = numeric()
    )

    for (pos in open_positions) {
      portfolio_summary <- portfolio_summary %>% add_row(
        Symbol = pos$contract$symbol,
        Position = pos$portfolioValue$position,
        Market_Price = round(pos$portfolioValue$marketPrice, 4),
        Avg_Cost = round(pos$portfolioValue$averageCost, 4),
        Market_Value = round(pos$portfolioValue$marketValue, 4),
        Unrealized_PnL = round(pos$portfolioValue$unrealizedPNL, 4)
      )
    }

    print(portfolio_summary)
  } else {
    cat("No open positions in portfolio.\n")
  }

  twsDisconnect(tws)
  cat("\nDisconnected from TWS.\n")
}

evaluate_portfolio_performance(IBport)
```

Result - Real-Time Portfolio Evaluation

	Symbol	Position	Market_Price	Avg_Cost	Market_Value	Unrealized_PnL
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	AAPL	400	211.	189.	84244	8734.
2	AFL	-74	105.	104.	-7789.	-82.4
3	BKNG	-9	5041.	5039.	-45372.	-16.8
4	BLK	-79	916.	902.	-72389.	-1128.
5	CPRT	147	60.9	61.0	8951.	-21.6
6	GL	-63	115.	112.	-7239.	-186.
7	HIG	-68	123.	121.	-8332.	-96.1
8	KEYS	50	146.	147.	7298.	-74.5
9	KLAC	-72	690.	699.	-49655.	679.
10	MCK	-12	709.	701.	-8511.	-94.9

This table captures the state of the portfolio after executing live trades, including ticker symbols, share counts, market prices, cost basis, and unrealized profits/losses

Result - Execution Confirmation in Interactive Brokers

- The display shows unfilled order statuses (e.g., 0/149 for CPRT), which may reflect pre-market conditions or order throttling by TWS at the time of placement.
- The visibility of both long and short signals confirms the system's support for directional trades
- Validates the integration between R and Interactive Brokers

Challenges Encountered

- Interactive Brokers API Limitations: Establishing a reliable connection with Interactive Brokers' Trader Workstation (TWS) was occasionally hindered by session conflicts, port availability issues, and API rate limits. Randomizing client IDs and implementing safe disconnects helped stabilize sessions

- Order Throttling and Market Timing: Some trades remained unfilled due to premarket or post-market timing. Orders submitted outside active trading windows were acknowledged but not executed. This made it difficult to simulate true production fill behavior

- Data Inconsistencies and Missing Values: Price and volume data from the Nasdaq SHARADAR feed occasionally included gaps or stale values for certain tickers. This required additional checks to filter out incomplete histories and pad missing indicators

Challenges Encountered

- Nasdaq API Rate Limits and Insider Data Challenges: Data collection through Nasdaq's SHARADAR API encountered rate limits, especially when retrieving insider trading data across multiple tickers in parallel

- Backtest vs Live Discrepancy: Discrepancies in return profiles between the backtest and live periods highlighted the sensitivity of the model to small changes in timing, execution latency, or signal thresholds. This emphasized the importance of post-deployment monitoring.

- Rolling Volatility and Position Sizing: Volatility spikes during live trading resulted in unintended leverage at times. This motivated the need for integrating dynamic volatility-based sizing in future iterations

Conclusion

Successfully demonstrated the development and deployment of a production ready equity trading strategy using R and the Interactive Brokers API. The strategy integrated classical technical indicators—Bollinger Bands and SMA crossovers—with robust signal scoring, capital allocation, and execution logic